

★第6章

エンジニアだけが開発生産性を求められる重圧

《登場人物》

湊太郎

田中部長

伊藤部長

山田健一

飛鳥さくら

佐藤

6-1

エンジニア視点と
ビジネス視点の融合

ストーリー

経営会議への招待

AI活用を始めてから半年が経過した。湊のチームは着実に成果を上げていた。

開発速度は30%向上し、バグ検出率は40%向上していた。技術的負債の返済も進み、チーム全体の疲労度も軽減されていた。これまでのAI活用の取り組みは、開発生産性ツリーの観点から整理すると、戦略1（開発プロセスの高速化）によりIncrement（出口）を1.5倍にし、戦略2（Ops工数のAI置き換え）によりCapacity（入口）を1.5倍にすることで、全体として開発生産性を150%に向上させることができていた。

そんなある日、田中部長から湊に呼び出しがあった。部長室に通され、湊は椅子に腰を下ろした。

「湊君、これまでのAI活用の成果、経営層も注目してるよ。他部署からの評価も良好だったし、開発生産性ツリーの観点から説明できるという話も聞いた」

田中部長の表情には、期待と緊張が入り混じっていた。

「来月の経営会議で、成果発表の機会を設けてもらえることになった。湊君、君が発表してくれるか？」

湊は一瞬、返答に困った。経営会議で発表する。それは大きなチャンスかもしれない。でもエンジニアの視点だけで説明して、本当に理解してもらえるだろうか。言葉がすぐには出てこなかった。

不安と期待が入り混じる

「エンジニアの視点だけで大丈夫かな？」

湊は一呼吸おいて、正直に不安を口にした。

「君なら、経営層にも分かるように説明できるはずだ。でも準備はしっかりしてほしい。他部署の声も聞いてみるといいかもしれない。何かあれば俺もフォローする」

田中部長の言葉に、湊は頷きながら聞いた。確かにエンジニアの視点だけでは不十分かもしれない。これまでに他部署からの評価は得られていた。営業部からは顧客満足度の向上、サポート部からはバグ報告の削減、マーケティング部からはリリース計画の安定化という評価を得ていた。

でもそれをどう経営層に説明すればいいのか。技術的な話だけでは、伝わらないかもしれない。頭のなかで言葉を組み立て直そうとするが、まだ形にならない。

伊藤部長との対話

湊は、田中部長の同席のもと事業責任者の伊藤部長との1on2ミーティングを設定した。会議室に三人が向かい合う。

「湊君、君のチームの取り組みは素晴らしいと思う。でも正直、最初は『また技術的な話か』と思ってたんだ」

伊藤部長は資料に目をやり、率直に語った。

「事業視点では、どう見えますか？」

湊は一呼吸置いて、勇気を出して質問した。

「うん。事業視点で見ると、いくつか評価できるポイントがある」

伊藤部長は資料を見ながら続ける。

「まず、顧客満足度の向上。バグが減ることで、お客様からのクレームが減った。これは直接的に事業価値につながっている。あと、うちはtoC向けのサービスもやっているから、リリース後の障害が減ったことで、障害でサービスが止まっている間の売上損失もなくなった」

「あとは、AI活用で生まれた余剰工数を使って、今まで手がかけられなかったサーバーインフラのコスト改善に取り組んでくれた。コストが下がって、売上に對する利益率が上がったのは良かった」

「次に、開発コストの最適化。技術的負債の返済により、開発速度が向上した。同じ機能を開発するのに、以前より少ない工数で済むようになった。これはコスト削減につながる」

「最後に、市場投入スピードの安定化。リリース計画が安定することで、マーケティングや営業の計画も立てやすくなった。これは競争優位性につながる」

湊は驚いた。技術的な改善が、これほどまでに事業価値につながっているとは思っていなかった。自分たちの日々の取り組みが、こういう言葉で語られるのだと腹落ちする。

「技術的改善が事業価値につながってるんですね」

「そうだ。でも実は、もっと根本的なことがある」

伊藤部長は資料をめくってから続ける。

「僕ら事業サイドが求めているのは、開発の速度じゃない。『この見積もりを信じて投資判断していいか』という確信だ。半年の計画を信じて営業戦略を立てられるか。それが全てだ。エンジニアはリードタイムやデプロイ頻度を見ている。でも事業責任者は、計画を信じて立てられるか、意思決定の前提として使えるかを見ている」

湊は真剣に聞き入った。

「今回の改善で、計画を信じて立てられるようになった。リリース計画が安定し、予測可能性が高まった。それが一番の価値だ。技術的な指標が改善しても、計画信頼性が向上しなければ、事業責任者には『良くなった実感』がない。でも今回は、その両方が実現できている」

「やっぱり、信頼性が開発生産性の本質なんですね」

「あと一つ、開発が追いつかないと言うとき、足りないのは実装リソース、つまり人数なのか、それとも『何を作るか』を決める力なのか。そこを間違えると、人を増やしても忙しいまま進まない。本当のボトルネックがどこにあるか、見極める必要がある」

伊藤部長がそう付け加えると、湊ははっとした。

「本当のボトルネックがどこにあるか、見極める。その視点は持っていませんでした」

「そうだ。でもそれをどう説明するかが重要だ。エンジニアの視点だけで説明しても、経営層には伝わらない。事業視点で説明することで、初めて理解してもらえる」

伊藤部長の言葉に、湊は静かに頷いた。確かにエンジニアの視点だけでなく、事業視点も必要だった。経営会議に向けて、何をどう語るか。その輪郭が少し見えてきた。

事業視点の理解

伊藤部長の言葉を噛みしめながら、湊は開発組織と事業側で測っているものが違うこと、計画信頼性が本質だと腹落ちした。

解説 エンジニア視点とビジネス視点の融合

この節の論点：エンジニア視点とビジネス視点で「測っているもの」が違うことを理解し、開発生産性の本質が約束の信頼性であることを共有する必要性です。

⑤ ストーリーで描かれる「重圧」

「経営層に開発生産性をどう説明するか」が重圧になるのは、**何が起きているかが関係者で共有されていない**ためです。湊が直面したのも、この構造から生じています。

- **経営層からの評価依頼** 「エンジニアの視点だけで大丈夫か」という不安。技術的な改善をどう伝えればよいか
- **これまでの成果の振り返り** 他部署からの評価を実感し、エンジニアの改善が会社全体に影響していることを改めて認識する
- **事業責任者との議論** 開発生産性の本質は約束の信頼性であること、開発組織が見ているものと事業責任者が見ているものは違うことを理解する
- **「計画を信じて立てられるようになったのが一番の価値だ」という評価**。信頼性が開発生産性の本質だと腹落ちする

この重圧の背景には、**エンジニア視点だけでは伝わらず、事業視点での「信頼性」を共有して初めて評価が伝わる構造**があります。両視点の融合の必要性に気づいた段階にあるのです。

⑤ 開発生産性の本質的な定義

湊が経験したように、エンジニア視点とビジネス視点の違いを理解し、融合させることで、共通言語を確立できます。

開発生産性の本質は第1章で触れた通り、約束の信頼性にある。

⑤ エンジニア視点とビジネス視点の違い

エンジニア視点

- 技術的品質、持続可能性、コードの複雑度

- 開発速度、バグ発生率、テストカバレッジ
- 技術的負債の管理、リファクタリングの重要性
- **測っているもの** リードタイム、デプロイ頻度、変更失敗率、MTTR（内部状態・流動性・安定性）

ビジネス視点

- 売上貢献、コスト削減、市場投入速度
- 顧客満足度、競争優位性、事業成長
- ROI、投資対効果、財務的な成果

事業/経営視点

- 持続可能な資産とお金、財務的な成果
- ソフトウェア資産計上、減価償却費、P/Lへの影響
- 長期的な競争力、組織全体の成長
- **測っているもの** 計画信頼性・予測可能性（欲しいタイミングで欲しいものが出るか、計画は信じて立ててよいか）

⑤ 測っているものが違う

開発組織が見ているもの（リードタイム、デプロイ頻度、変更失敗率、MTTR）と、事業責任者・PdMが見ているもの（計画信頼性・予測可能性）は異なります。Four Keysなどの指標は開発組織の内部安定性を測りますが、事業計画の信頼性は直接測りません。これが、Four Keysが改善しているにもかかわらず、事業責任者・PdMには「良くなった実感」がないギャップの原因です。

6-2

共に創る
「共通の価値基準」

ストーリー

共通言語を探す

経営会議でのプレゼンに向けて、湊はチームメンバーとプレゼン内容を検討していた。いつもの会議室で、ホワイトボードの前にメンバーが集まる。

共通言語の模索

「技術用語をビジネス用語に翻訳する必要があるね」

山田が腕を組みながら提案した。

「例えば、技術的負債をどう説明するか。経営層に『技術的負債』と言っても、理解してもらえないかもしれない」

湊はホワイトボードに書きながら考えた。

「技術的負債は、開発効率の低下リスクとして説明する。どうかな？」

「それだと、まだ抽象的かもしれない。もっと具体的に、コストや時間で説明できないか？」

飛鳥が身を乗り出して加わった。

「リファクタリングは、将来への投資として説明する。これはどう？」

「いいね。でも投資対効果も示した方がいいかもしれない」

山田が補足した。

「コードの品質は、継続的な競争優位性として説明できる。これは事業視点で説明できそうだね」

山田が腕を組んでから続ける。

「そういえば、以前のワークショップで学んだISO/IEC 25010の品質特性も、共通言語として使えるかもしれない。品質を8つの特性に分類することで、経営層にも多面的に説明できる。例えば、保守性の向上が開発速度の向上につながることも、信頼性の向上が顧客満足度の向上につながることなど」

湊は頷く。

「確かに。品質を単一の指標で測るのではなく、多面的に評価することで、経営層にも理解してもらいやすくなるかもしれない」

湊は書きながら続ける。

「技術的負債の返済により、開発速度が年間20-40%向上。これはコスト削減につながる」

「バグ修正コストをリリース後の30-100倍から開発時の10倍に削減。これも具体的な数値で示せる」

飛鳥が頷く。

「これなら経営層にも伝わりそう。技術的な話を、事業価値に結びつけて説明する。それが重要だね」

飛鳥は少し考えてから続けた。

「伊藤部長との話で気づいたんだけど、『人が足りない』と言われたとき、足りないのは人数ではなく、優先順位をつけて『これは作る、これは作らない』と決められる力かもしれない。その視点も、経営層に伝えられるとよいかもしれない」

山田も同意した。

「技術的な補足も必要だけど、まずは事業価値で説明する。その後に、技術的な詳細を説明する。その順序が重要だと思う」

湊はチームメンバーの意見を聞きながら、プレゼン資料を作成していった。

開発生産性ツリーの可視化

湊はホワイトボードの前に立ち、新しい図を描き始めた。

「実は、もう一つ説明したいことがあります。田中部長と話していたのですが開発投資がどこに使われているかを、構造的に見える化する方法です」

湊はホワイトボードに矢印と箱を描きながら続ける。

「予算から始まって、人件費に変換され、開発区分に配分されます。そして、工数、施策、バリューストリーム、開発ライフサイクルまで、一本の線でつながっています」

山田が興味深そうに見つめた。

「これは面白い。予算という財務の視点から、実際の開発投資まで接続できるんだね」

「はい。そして、ここが重要なんですが」

湊は開発区分の部分了指差した。

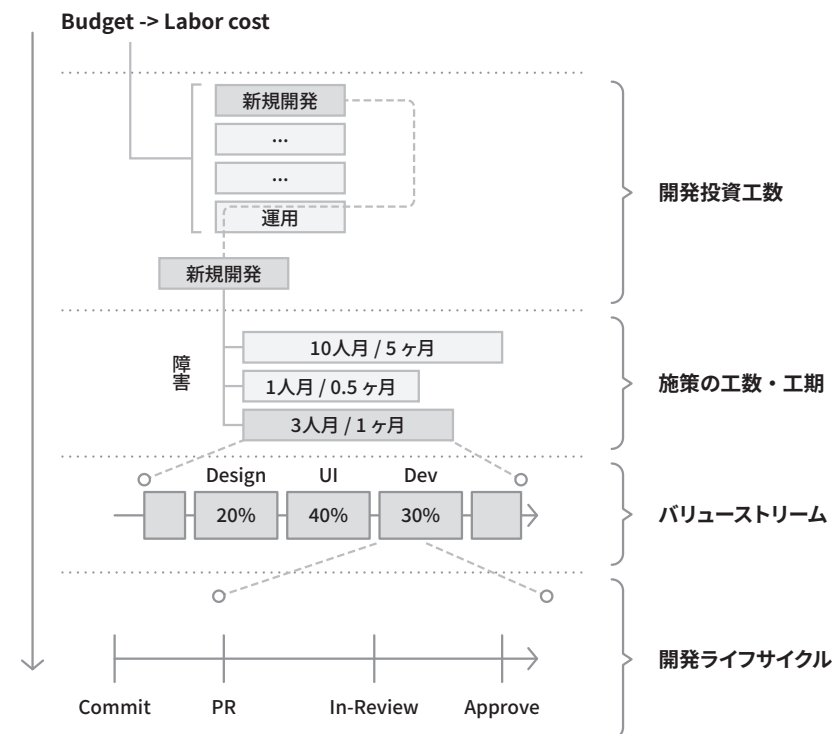


図 6.1: 開発生産性ツリー：予算から人件費、開発区分へとつながり、工数・施策・バリューストリーム・開発ライフサイクルまで一本の流れで可視化するイメージ

「開発区分は、新しい価値を作る工数と現状システムを維持する工数に分けられます。新規開発やエンハンス開発は前者、保守開発や運用、管理業務は後者です」

飛鳥が頷く。

「これなら、どこにお金と時間が使われているか一目瞭然ですね。経営層にも『なぜ開発生産性が低いのか』が伝わるはずです」

山田が補足した。

「この構造を使えば、投資対効果も説明しやすくなる。予算がどの施策に、どの工程で、どの開発フェーズで消費されているかが明確になるからね」

湊はチームメンバー全員が納得しているのを確認し、プレゼン資料に開発生産性ツリーの図を追加した。

経営会議の前夜

経営会議当日。大きな会議室に役員 10 名が並び、湊は壇上に立った。緊張しながらも、準備してきた資料を説明し始めた。

経営会議での緊張のプレゼン

「開発生産性向上の取り組み報告」

湊はデータと事例を交えながら、20 分間の発表を行った。

「まず、成果からご報告します。開発速度が 30% 向上し、バグ検出率が 40% 向上しました。これにより、顧客満足度が向上し、サポート対応の工数も削減できました」

経営層の反応は、最初は懐疑的だった。でも湊が具体的な数値と事例を示すにつれて、徐々に真剣な表情に変わっていった。会議室の空気が、少しずつ湊の説明に傾いていく。

湊は画面を切り替え、開発生産性ツリーの図を映した。

「次に、開発投資がどこに使われているかの構造です。先ほど田中部長とお話していた、予算から開発ライフサイクルまで一本でつながる枠組みで整理しています。新しい価値を作る工数と現状システムを維持する工数の区別で見ると、現在は現状システムを維持する工数に時間を使っている状況でした」

湊は AI 戦略の説明に移った。

「これまでの AI 活用の取り組みは、開発生産性ツリーの観点から 2 つの戦略に整

理できます。戦略 1 は、開発プロセスの高速化により、同じ投入工数でより多くの成果物を生み出すことで、Increment（出口）を 1.5 倍にします。戦略 2 は、Ops 工数を AI に置き換えることで、余剰工数を新規開発に回し、Capacity（入口）を 1.5 倍にします。両戦略の組み合わせにより、開発生産性を 150% に向上させることができます」

「技術的負債の返済により、開発速度が年間 20-40% 向上しています。これは同じ機能を開発するのに、以前より少ない工数で済むことを意味します。開発コストの削減につながっています」

「バグ修正コストをリリース後の 30-100 倍から開発時の 10 倍に削減しました。これにより、バグ対応の工数が大幅に削減され、新機能開発に集中できるようになりました」

CEO が質問した。

「つまり短期的なスピードよりも、持続可能性が大事ということか」

「はい、長期的な事業成長のための基盤作りです。短期的には開発速度が少し低下するように見えるかもしれませんが、長期的には開発速度が向上し、コスト削減につながります」

湊は一息ついて、自信を持って答える。

経営層との質疑応答

CFO が手を挙げた。湊は緊張しながら、CFO の方を向く。

「投資対効果はどの程度？」

CFO の声は冷静だった。湊は深呼吸をして、準備してきた資料を見せながら答える。ここは何度も想定してきた質問だ。

「開発コスト 15% 減、バグ対応工数 40% 減です。技術的負債の返済に投資した時間は全体の 20% 程度ですが、その投資により開発速度が年間 20-40% 向上しています。投資対効果は約 2 倍以上と試算しています」

湊は画面にグラフを映しながら続ける。

「具体的には、技術的負債の返済により、開発速度が年間 20-40% 向上。バグ修正コストをリリース後の 30-100 倍から開発時の 10 倍に削減。これにより、年間で約 500 万円のコスト削減が見込めます」

CFO は資料を見つめながら、少し考え込んだ様子だった。でも数値に納得したのか、小さく頷く。

CTO が続けて質問した。

「他のチームにも展開できるの？」

CTO の声には期待が込められていた。湊は少し緊張しながらも、用意していた答えを口にした。

「まずノウハウを体系化して、段階的に展開する計画です。今回の取り組みで得た知見を他のチームにも共有していきます。具体的には、技術的負債の可視化方法、リファクタリング時間の確保方法、AI 活用のベストプラクティスなどを文書化して、他チームに展開する予定です」

CTO は満足そうに頷く。

「それは良い。ぜひ進めてほしい」

CEO も湊を見て言う。

「湊さん、素晴らしい取り組みだ。技術的な改善が、ビジネス価値につながる。それを数値で証明できたことが重要だ。この取り組みを全社標準にしたい」

湊は予想以上の好反応に驚いた。経営層が技術的な改善の価値を理解してくれている。それはエンジニアの視点だけでなく、事業視点で説明したからだ。湊はこの時、ビジネス視点を持つことの重要性を実感した。技術的な改善をビジネス価値に変換して説明することで、経営層にも理解してもらえる。それが、エンジニアリーダーとしての成長だった。会議室を出た後も、その手応えが胸に残っていた。

会議室を出た湊に、山田が耳打ちした。

「聞いた話だけど、部長が事前に CFO と CTO に個別に説明していたらしい。『湊のチームの取り組みを聞いてやってほしい』って」

湊は驚いた。田中部長は何も言っていなかった。数字を出せと迫っていたあの部長が、陰で湊を支えていた。

解説 共に創る「共通の価値基準」

技術用語をビジネス用語に翻訳し、開発生産性ツリーで財務と開発投資を接続して、共通の価値基準を創る実践です。

⑤ ストーリーで描かれる「重圧」

データはあるが伝え方がわからない。そんな状態のとき、「共通の価値基準を創る」はとりわけ重くのしかかります。湊のチームが経験した重圧も同じ構造からです。

- **技術用語をビジネス用語に翻訳する作業** 技術的負債を開発効率の低下リスクとして、リファクタリングを将来への投資として説明する
- **開発生産性ツリーで予算から開発ライフサイクルまでの因果を可視化** 財務と開発投資を接続し、投資がどこでどう消費されているかを示す
- **経営会議での緊張のプレゼン** データと事例に基づく説明で、技術的改善の価値を事業視点で伝える
- **CFO からの投資対効果の質問に数値で答える** エンジニアとビジネス側の共通理解が深まる手応えを得る

この重圧の背景には、専門用語と価値基準のズレを解消し、共通言語と可視化で初めて相互理解が進む構造があります。共通の価値基準を創る実践段階にあるのです。

⑥ なぜ共通言語が必要なのか

湊が実践したように、共通言語を確立することで、エンジニアとビジネス側の相互理解が深まります。

⑥ 開發生産性はレイヤーごとに意味が違う

開發生産性という語は、組織の階層によって指すものがまったく異なります。以下の図をご覧ください。

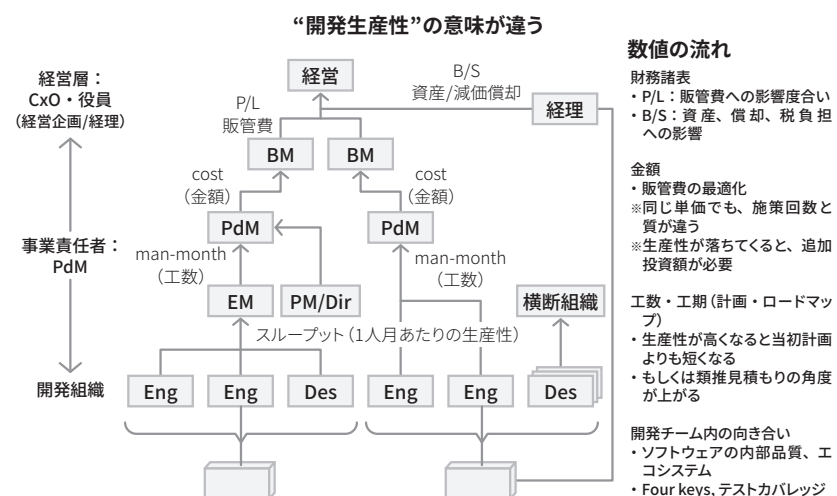


図6.2：「開發生産性」の意味が違う：開発組織のスループットから人月・金額へと変換され、経営・経理では損益計算書の販管費と貸借対照表の資産・減価償却などに接続するイメージ

開発チームの現場では、開發生産性はスループット、すなわち1人月あたりにどれだけ成果物を出せるかをさします。ソフトウェアの内部品質やテストカバレッジ、デプロイ頻度などを含むFour Keysのような指標がここに対応し、エンジニアやデザイナーが日々扱う領域です。

一段上のエンジニアリングマネージャー（EM）やプロジェクトマネージャー

（PM）、ディレクター層では、スループットは人月に換算されます。ここでの開發生産性は、計画やロードマップに対して工数や工期がどう動くかという問いです。開發生産性が高いほど当初計画より短い工期で済み、見積りの精度も上がりやすくなります。

さらに上の事業責任者であるプロダクトマネージャー（PdM）やビジネスマネージャー（BM）の層では、人月は金額として読み替えられます。販管費の最適化が焦点になり、同じ単価でも開發生産性次第で施策の回数や質が変わります。開發生産性が落ちれば、同じ成果を得るのに追加の投資が要ります。

最上位の経営層では、金額は損益計算書や貸借対照表の語彙へと移ります。損益計算書上は販管費としてどれだけ影響するか、貸借対照表上は資産計上や減価償却、税負担にどう効くかが論点になり、経理部門は貸借対照表側の数値を扱います。

現場がスループットを上げたと言うとき、経営が聞いているのは販管費が下がったのか、資産効率は改善したのか、という問いです。同じ開發生産性という一語でも、階層ごとに通貨の単位が違います。この変換の連鎖を意識しないまま改善を語ると、現場の努力が経営の言葉に届かず、投資判断や評価のすれ違いが起きます。

こうした開發生産性という言葉が各階層の数値がどうつながっているかを可視化し、共通言語を持つことが、開發生産性の議論を実りあるものにする出発点です。

6-3

開発生産性を
評価制度に組み込む

ストーリー

新しい評価制度の提案

経営会議の後、経営層から要請があった。

「この取り組みを制度化できないか？ 他のチームにも展開してほしい」

湊は山田と飛鳥と共に、新しい評価制度の設計を始めた。

会議室で、3人はホワイトボードの前に集まった。経営会議の余韻がまだ残るなか、次の一步を具体化する段階だ。

「まず5名チーム全員からフィードバックを収集しよう」

湊はチームメンバーに意見を求めた。

「評価制度って、どういうものがいいと思う？」

佐藤が少し考えてから最初に発言した。

「工数や工期だけでなく、品質も評価してほしい。バグが少ない方が結果的には効率的だと思うから。でも今の評価制度だと、バグが少なくても評価されない。むしろ、バグを出さないために時間をかけたら、評価が下がってしまう」

佐藤の言葉に、他のメンバーも頷く。

山田も続ける。

「技術的負債の管理も評価してほしい。短期的な速度だけで評価されると技術的負債が蓄積してしまう。ワークショップで学んだように、形骸化した指標に依存すると、本質的な改善につながらない。リファクタリングの時間を確保しても、それが評価されないと、誰もやらなくなる」

飛鳥も意見を述べた。

「事業価値への貢献も評価してほしい。技術的な改善がどう事業に貢献しているかを可視化したい。エンジニアの視点だけでなく、PMや事業責任者の視点も取り入れた評価軸が必要だと思います」

湊はメンバーの意見を聞きながら、ホワイトボードに書き出していった。個人がどれだけ速く成果を出しても、それが組織の数字や事業KPIにそのまま表れるとは限らない。だからこそ、評価軸そのものを多角的に設計する必要がある、と湊は考えた。

「じゃあ、3つの軸で評価する制度を設計してみましょう。ビジネスインパクト、持続可能性、効率性。この3つで評価すれば、多角的な開発生産性評価ができると思います」

山田が頷く。

「いいね。でもそれぞれの軸で、どういう項目を評価するのか、もっと具体的に決める必要があるね」

飛鳥も同意した。

「そうですね。例えば、ビジネスインパクトなら、顧客満足度の向上、売上への貢献、バグ対応工数の削減など。持続可能性なら、技術的負債の管理状況、コード品質の維持・向上、チームメンバーの成長など。効率性なら、開発速度、バグ発生率、テストカバレッジなど」

湊はメンバーの意見を聞きながら、3つの軸で評価する制度を設計した。

1. ビジネスインパクト（売上・顧客満足度への貢献）

- 顧客満足度の向上
- バグ対応工数の削減
- 市場投入速度の安定化

2. 持続可能性（技術的負債管理、チーム成長）

- 技術的負債の管理状況
- コード品質の維持・向上
- チームメンバーの成長

3. 効率性（開発速度、品質）

- 開発速度の向上
- バグ発生率の削減
- テストカバレッジの向上

「全ステークホルダーが理解できる指標にしたい」

湊は設計した評価制度を説明した。

「エンジニアの視点だけでなく、PM、事業責任者の視点も取り入れた評価軸です。これにより、多角的な開発生産性評価が可能になります」

組織文化の変化の始まり

新しい評価制度の試験導入が始まった。

他チームからの関心も高かった。廊下ですれ違う他チームのリーダーから、声をかけられることも増えた。

「うちでもやってみたい。どうやって始めればいい？」

「まずチーム全体で現状を把握することから始めます。その後、段階的に改善を進めていきます」

湊は他チームからの質問に答えながら、組織全体への波及効果を実感していた。

田中部長が湊を呼び出した。部長室で、田中部長は満足そうな表情を浮かべている。

「湊君、君が変化の起点になったね。この取り組みが、組織全体に良い影響を与えている。前は進捗会議で数字だけ見てたけど、今は動いているところを見に来るのが楽しみになってる。出てくるものに一貫性が出てきたので任せやすくなった」

「一人では無理でした。チーム全体の力です」

湊は少し照れくさそうに、正直に答える。

山田も湊を評価していた。

「湊くん、君が切り開いた道を、他のチームも歩み始めてるね。エンジニアの視点だけでなく、事業視点も持つ。それが重要だったんだよ」

佐藤らチームメンバーも成長していた。技術的なスキルだけでなくビジネス視点も持つようになっていた。会議での発言の仕方や、他部署との話し方にも、その変化が表れていた。

「エンジニアだけの問題」から「組織全体の取り組み」へ。湊はその変化を実感していた。

手法 9：多角的開發生産性評価マトリクス

評価軸の設計

多角的な開發生産性評価を行うためには、3つの軸で評価すると、バランスの取れた判断ができます。

1. ビジネスインパクト

評価項目

- 顧客満足度の向上（NPS、顧客満足度調査）
- 売上への貢献（機能別売上分析）
- バグ対応工数の削減（サポート対応時間の削減）
- 市場投入速度の安定化（リリース計画の遵守率）

測定方法

- 定量的指標 NPS、売上データ、サポート対応時間
- 定性的評価 顧客フィードバック、営業・サポートからの評価

2. 持続可能性

評価項目

- 技術的負債の管理状況（技術的負債スコア、複雑度）
- コード品質の維持・向上（テストカバレッジ、コードレビュー）
- チームメンバーの成長（スキルマップ、学習時間）
- ドキュメント整備状況（ドキュメントカバレッジ）

測定方法

- 定量的指標 技術的負債スコア、テストカバレッジ、スキルマップ
- 定性的評価 コードレビュー、チームメンバーの成長実感

3. 効率性

評価項目

- 開発速度（デプロイ頻度、変更のリードタイム）
- 品質（バグ発生率、変更失敗率）

- コスト効率（開発コスト、バグ修正コスト）

測定方法

- 定量的指標 デプロイ頻度、リードタイム、バグ発生率
- 定性的評価 チームの疲労度、開発のしやすさ

評価の実施方法

ステップ1 現状の把握

各軸の評価項目について、現状を把握します。定量的な指標と定性的な評価を組み合わせ、総合的に評価します。

ステップ2 目標の設定

各軸について、具体的な目標を設定します。目標は、SMART（Specific, Measurable, Achievable, Relevant, Time-bound）の原則に従って設定します。

ステップ3 定期的なレビュー

四半期ごとに評価を実施し、進捗を確認します。必要に応じて、目標や評価項目を見直します。

DORA Metricsの具体的な活用方法

多角的な開發生産性評価において、DORA Metricsを活用する具体的な方法を以下に示します。

DORA Metricsの測定方法

1. デプロイ頻度（Deployment Frequency）

- 測定 本番環境へのリリース回数を期間で割る（例）週1回、月1回
- 目標設定 エリートレベル（1日1回以上）、高パフォーマンス（週1回～月1回）
- ビジネス指標との関連 市場投入速度、競争優位性

2. 変更のリードタイム（Lead Time for Changes）

- 測定 コードコミットから本番デプロイまでの時間を測定

- **目標設定** エリートレベル（1時間未満）、高パフォーマンス（1日～1週間）
- **ビジネス指標との関連** 開発コスト削減、ROI向上

3. 変更失敗率（Change Failure Rate）

- **測定** 本番デプロイ後に障害が発生した回数を全デプロイ回数で割る
- **目標設定** エリートレベル（0-15%）、高パフォーマンス（16-30%）
- **ビジネス指標との関連** 顧客満足度、サポートコスト削減

4. サービス復元時間（Time to Restore Service）

- **測定** 本番障害の発生から復旧までの時間を測定
- **目標設定** エリートレベル（1時間未満）、高パフォーマンス（1日未満）
- **ビジネス指標との関連** 事業継続性、信頼性向上

⚙️ DORA Metrics とビジネス指標の統合レポート

定期的に DORA Metrics とビジネス指標を統合したレポートを作成することで、技術的な改善がビジネス価値にどうつながるかを明確に示すことができます。

レポート例

- デプロイ頻度が週1回から1日1回に向上した結果、市場投入速度が30%向上し、売上に10%貢献した事例
- 変更失敗率が20%から10%に削減した結果、サポート対応時間が40%削減し、コスト削減500万円/年につながった事例

このような形で、DORA Metrics をビジネス指標と関連付けることで、エンジニア視点とビジネス視点の橋渡しが可能になります。

出典：DORA（DevOps Research and Assessment）レポート、
『Accelerate: The Science of Lean Software and DevOps』

評価結果の活用

評価結果は、以下の目的で活用します。

- **チームの成長** 評価結果を基に、チームの強みと弱みを把握し、改善計画を立てる
- **意思決定の支援** 評価結果を基に、技術投資の優先順位を決定する
- **組織全体の改善** 評価結果を共有し、組織全体の改善に活用する

手法 10：ステークホルダー対話ガイド

対話の目的

エンジニアとビジネス側の対話を促進することで、相互理解を深め、より建設的な意思決定ができるようになります。

対話の準備

⚙️ ステップ1 相手の視点を理解する

対話の前に、相手の立場や関心事を理解します。エンジニアの視点とビジネス視点の違いを認識し、共通言語を準備します。

⚙️ ステップ2 目的を明確にする

対話の目的を明確にします。技術的な課題を説明するのか、投資判断を求めるのか、優先順位を決定するのか。目的に応じて、準備する資料や説明方法を変えます。

🔧 ステップ3 資料の準備

技術的な内容を、ビジネス用語で説明できる資料を準備します。具体的な数値や事例を含め、視覚的に分かりやすい資料を作成します。

対話の進め方

🔧 1. 共通言語の確立

技術用語をビジネス用語に翻訳します。

- 技術的負債：開発効率の低下リスク
- リファクタリング：将来への投資
- コードの品質：継続的な競争優位性

🔧 2. 事業価値での説明

技術的な改善を、事業価値に結びつけて説明します。

- 開発速度の向上：コスト削減、市場投入速度の向上
- バグ発生率の削減：顧客満足度の向上、サポート対応工数の削減
- 技術的負債の返済：長期的な開発速度の向上、コスト削減

🔧 3. 具体的な数値での説明

定性的な説明だけでなく、具体的な数値で説明します。

- 開発速度が30%向上した場合：開発コスト15%削減
- バグ検出率が40%向上した場合：バグ対応工数40%削減
- 技術的負債の返済：開発速度が年間20-40%向上

定期的な対話の場

🔧 Tech & Business Sync ミーティング

月次で実施し、エンジニアとビジネス側が技術戦略を議論します。優先度を透

明性高く決定し、相互理解を深めます。

🔧 四半期レビュー

四半期ごとに、開発生産性の状況を全レイヤーで共有します。評価結果を基に、改善計画を立てます。

🔧 投資判断の透明化

開発生産性向上への投資判断を透明にし、全ステークホルダーが納得できる判断をします。

対話の効果

定期的な対話により、以下の効果が得られます。

- 相互理解の促進 各ステークホルダーの視点を理解できる
- 意思決定の改善 共通言語で対話することで、より建設的な意思決定ができる
- 組織全体の成長 エンジニアとビジネス側が協働することで、組織全体の開発生産性が向上する

章のまとめ

本章では、湊がエンジニアの視点だけでなく、PM、PdM、事業責任者の視点も取り入れた新しい開発生産性評価の枠組みを提案する過程を描きました。伊藤部長が語ったように、開発生産性の本質は約束の信頼性にあり、計画を信じて立てられるようになることが事業側にとっての一番の価値だという理解が、経営会議と新評価制度の設計に貫かれています。

🔧 主な学び

1. エンジニア視点とビジネス視点の融合 各レイヤーで「開発生産性」の意味が

異なることを理解し、共通言語を確立する

2. **共通の価値基準の確立** 技術用語をビジネス用語に翻訳し、事業価値に結びつけて説明する
3. **多角的な開発生産性評価** ビジネスインパクト、持続可能性、効率性の3つの軸で評価する
4. **ステークホルダー間の対話** 定期的な対話により、相互理解を深め、より建設的な意思決定ができる

🌀 次章への展望

本章で新しい開発生産性評価の枠組みを描いた。次の章では、湊が開発生産性をあげるだけのエンジニアからの脱却を果たし、技術者からビジネス視点も持つエンジニアリーダーへ成長する過程を描く。

参考文献

本章で言及した文献・出典を挙げる。

- **DORA / Four Keys** DORA metrics guide: <https://dora.dev/guides/dora-metrics/>, Four Keys: <https://cloud.google.com/blog/products/devops-sre/using-the-four-keys-to-measure-your-devops-performance>
- **Noda, A. et al.** (2023). "DevEx: What Actually Drives Productivity?" ACM Queue, Vol. 21, No. 2. 知覚的指標とワークフロー指標の併用。
- **DORA 2025** State of AI-Assisted Software Development (2025): <https://dora.dev/research/2025/dora-report/>, Google Cloud 抄訳 (日本語): <https://cloud.google.com/blog/ja/products/ai-machine-learning/announcing-the-2025-dora-report>
- **Beller et al.** Beller et al. (2025) "What's DAT? Diff Authoring Time at Meta", <https://arxiv.org/pdf/2503.10977>
- **McKinsey** McKinsey "Yes, you can measure developer productivity" および "The state of AI in 2025" (McKinsey 公式サイトで検索)
- **NBER / 個人と組織のギャップ** NBER "Firm Data on AI" 等に基づく、個人レベルの開発生産性向上と組織レベルの成果のギャップに関する知見。IT 生産性パラドックスと歴史的教訓 (Solow, Brynjolfsson 等) については第7章末の参考文献「§7 (組織変革・補完的導入)」を参照。
- **石垣雅人.**『開発生産性を上げる改善』って儲かるの？ に答えられるようにする』. Speaker Deck, 開発生産性カンファレンス 2024, 2024/06/28. https://speakerdeck.com/i35_267/is-development-productivity-profitable スライド10「価値創造への主体性と開発生産性」を参照。スライド内で引用されている広木氏「大規模言語モデル時代の開発生産性」に基づく付加価値生産性・物的生産性 (回転数)・曝露の議論。